



A Project Report

on

SIMPLE CALCULATOR

Submitted in partial fulfilment of requirements for the award of the course

of

CGB1121 – DATA STRUCTURES

Under the guidance of

Mrs. S. PRABAVATHI M.E.,

Assistant Professor/CSE

Submitted By

PATHMASRI V.M (927625BCS116)

VISHALINI.S (927625BIT120)

JANASRUTHI VS (927625BSC017)

DEPARTMENT OF FRESHMAN ENGINEERING

M.KUMARASAMY COLLEGE OF ENGINEERING

(Autonomous)

KARUR – 639 113

MAY 2026

M. KUMARASAMY COLLEGE OF ENGINEERING

(Autonomous Institution affiliated to Anna University, Chennai)

KARUR – 639 113

BONAFIDE CERTIFICATE

Certified that this project report on “**SIMPLE CALCULATOR**” is the Bonafide work of **PATHMASRI V.M (927625BCS116), VISHALINI.S (927625BIT120), JANASRUTHI VS (927625BSC017)**, who carried out the project work during the academic year 2025 - 2026 under my supervision.

Signature

Mrs. S. PRABAVATHI, M.E.,
SUPERVISOR,

Department of Computer Science and Engineering,
M. Kumarasamy College of Engineering,
Thalavapalayam, Karur -639 113.

Signature

Dr. K. CHITIRAKALA, M.Sc., M.Phil., Ph.D.,
HEAD OF THE DEPARTMENT,

Department of Freshman Engineering,
M. Kumarasamy College of Engineering,
Thalavapalayam, Karur -639 113.

Submitted for the End Semester Review held on _____

INTERNAL EXAMINER

VISION AND MISSION OF THE INSTITUTION

VISION OF THE INSTITUTION

To emerge as a leader among the top institutions in the field of technical education

MISSION OF THE INSTITUTION

- Produce smart technocrats with empirical knowledge who can surmount the global challenges
- Create a diverse, fully-engaged, learner-centric campus environment to provide quality education to the students
- Maintain mutually beneficial partnerships with our alumni, industry, and Professional associations

PROGRAM OUTCOMES (POs)

Engineering students will be able to:

PO1: Engineering knowledge: Apply knowledge of mathematics, natural science computing, engineering fundamentals and an engineering specialization to develop to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development

PO3: Design/development of solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change

ABSTRACT

The Data Structure Based Smart Calculator is an application designed to perform both basic arithmetic calculations and data structure operations. The system allows users to perform mathematical operations such as addition, subtraction, multiplication, and division. In addition, it demonstrates important data structure concepts queue. while the queue data structure is used for enqueue and dequeue operations. The application is developed with a graphical user interface to make it easy and interactive for users. This project helps students understand how data structures can be applied in real-world applications like calculators and data

Processing systems

ABSTRACT WITH POs MAPPING

ABSTRACT	POs MAPPED
The Data Structure Based Smart Calculator is an application designed to perform both basic arithmetic calculations and data structure operations. The system allows users to perform mathematical operations such as addition, subtraction, multiplication, and division. In addition, it demonstrates important data structure concepts queue. while the queue data structure is used for enqueue and dequeue operations. The application is developed with a graphical user interface to make it easy and interactive for users. This project helps students understand how data structures can be applied in real-world applications like calculators and data processing systems.	PO1(3) PO2(2) PO3(2) PO4(1) PO5(3) PO6(1) PO7(1) PO8(2) PO9(2) PO10(1) PO11(3)

Note: 1- Low, 2-Medium, 3- High

SUPERVISOR

HEAD OF THE DEPARTMENT



TABLE OF CONTENTS

CHAPTER No.	TITLE	PAGE No.
1	Introduction	1
	1.1 Introduction	1
	1.2 Objective	1
	1.3 Data Structure Choice	2
2	Project Methodology	3
	2.1 Proposed work	3
	2.2 Block Diagram	5
3	Modules	6
	3.1 Input Module	6
	3.2 Queue Module	6
	3.3 Enqueue Module	6
	3.4 Dequeue Module	6
	3.5 Operational Module	6
	3.6 Output Module	7
4	Results and Discussion	8
5	Conclusion	11
	References	12
	Appendix	13

CHAPTER 1

INTRODUCTION

1.1 Introduction

A calculator is a commonly used tool for performing mathematical operations such as addition, subtraction, multiplication, and division. In computer science, calculators can also be designed using programming concepts and data structures to perform calculations efficiently. Data structures help organize and manage data in a systematic way, making computation faster and more reliable. The Data Structure Based Smart Calculator is a software application developed that performs both basic arithmetic operations and demonstrates important data structure concepts. The system allows users to perform calculations while also understanding how data structures queue work in practical applications. In this project, Queue operations such as enqueue and dequeue can also be demonstrated to show the concept of data processing order. The calculator is designed with a graphical user interface (GUI) to make it simple, interactive, and easy for users to operate. This project helps students understand how theoretical concepts of data structures can be applied in real-world applications. Here we have used queue to implement and carry out the process. It also improves programming skills and demonstrates the integration of data structures, algorithms, and graphical interfaces in building useful software applications.

1.2 Objective

Our objective is to develop a smart calculator that performs basic arithmetic operations by implementing the Queue data structure for processing inputs in FIFO order by demonstrating enqueue and dequeue operations in a practical application,

by designing a simple and user-friendly Graphical User Interface (GUI) to enhance the understanding of data structures through real-world implementation.

1.3 Data Structure Choice

When considering data structures:

- **Queue (Primary Data Structure):** The main data structure used in this project is a queue. It follows the First in First Out (FIFO) principle, where the first element inserted is the first one to be removed. It is used to store the input values temporarily before processing.
- **Array (Implementation of Queue):** The queue is implemented using an array in JavaScript. The array stores the input values, and built-in functions are used to perform queue operations.

CHAPTER 2

PROJECT METHODOLOGY

2.1 QUEUE

The Simple Calculator is a software application designed to perform basic arithmetic operations such as addition, subtraction, multiplication, and division. In this project, the calculator is developed using programming concepts along with the implementation of a queue data structure. The queue helps in managing and processing user inputs in a sequential manner following the FIFO (First In First Out) principle. The system is designed with a graphical user interface (GUI) that allows users to easily enter inputs and view results. The working of the calculator involves taking user input, storing it in the queue using enqueue operation, processing the data step-by-step, and then displaying the result after performing the required operation. The dequeue operation is used to remove elements from the queue once they are processed.

Key Features of Project Methodology

1. Sequential Processing:

The calculator processes inputs in an ordered manner using the queue, ensuring proper flow of operations.

2. Queue Implementation:

Queue operations such as enqueue and dequeue are used to manage input data efficiently.

3. User-Friendly Interface:

The graphical user interface (GUI) makes the calculator simple, interactive, and easy to use.

4. Dynamic Operation Handling:

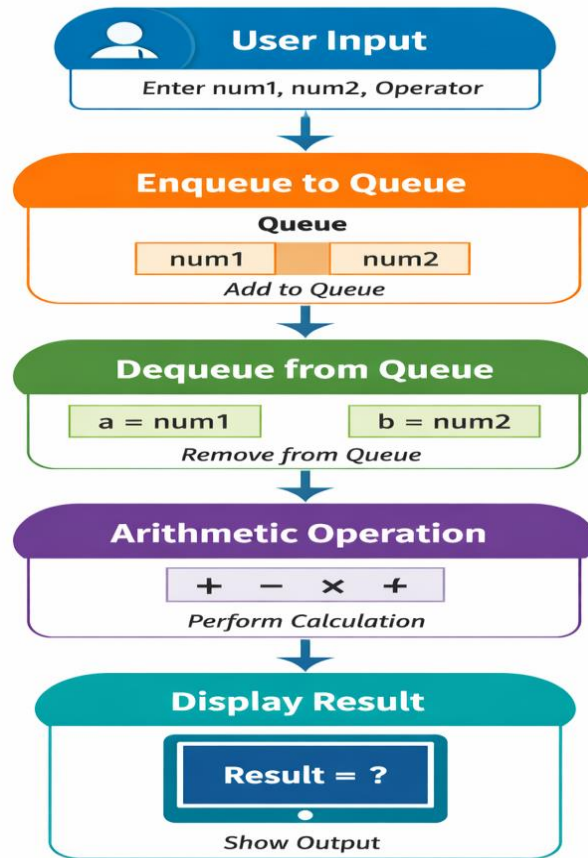
The system can handle different arithmetic operations dynamically based on user input

5. Efficient Execution:

The use of data structures improves the performance and organization of the calculation process.

In this project, operations such as input handling, processing, and output generation are carried out systematically using queue concepts. The methodology ensures that the calculator not only performs calculations but also demonstrates the practical application of data structure

2.2 Block Diagram



CHAPTER 3

MODULES

3.1 Input Module:

This module is responsible for accepting input from the user. It takes two numerical values (num1 and num2) and an arithmetic operator such as addition, subtraction, multiplication, or division through the user interface.

3.2 Queue Module:

This module manages the queue data structure. It initializes and maintains the queue, ensuring that elements are stored and processed according to the First In First Out (FIFO) principle.

3.3 Enqueue Module:

This module inserts the input values into the queue. The numbers entered by the user are added to the queue in sequence using the enqueue operation

3.4 Dequeue Module:

This module removes elements from the queue. The first inserted value is removed first, followed by the next value, and these are assigned to variables for further processing.

3.5 Operation Module:

This module performs the arithmetic calculation. Based on the selected operator, it applies the appropriate operation (addition, subtraction, multiplication,

or division) on the dequeued values.

3.6 Output Module:

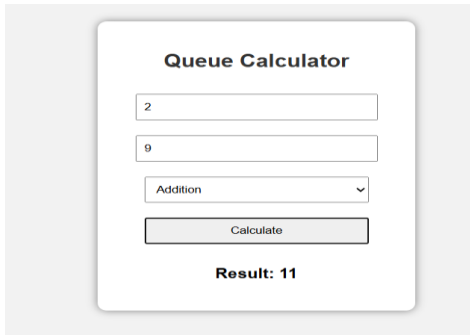
This module displays the result of the computation. The final output is shown to the user on the screen in a clear and understandable format.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Results

4.1.1 Addition operation



Queue Calculator

2

9

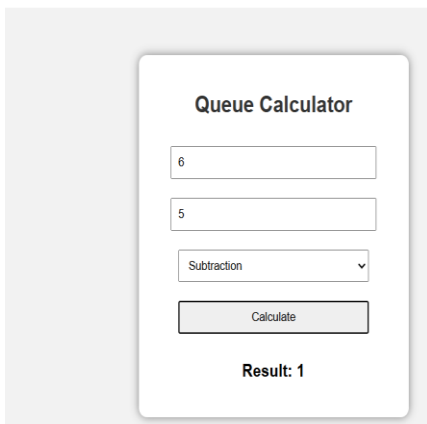
Addition

Calculate

Result: 11

Fig 4.1.1: This figure shows a calculator application designed to perform arithmetic operations addition. The user enters two numbers, selects the Addition operation, and clicks the Calculate button. The computed result is then displayed on the screen.

4.1.2 Subtraction operation



Queue Calculator

6

5

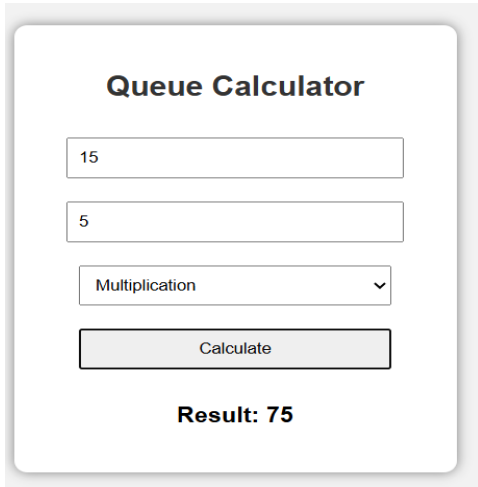
Subtraction

Calculate

Result: 1

Fig 4.1.2: This figure shows a calculator application designed to perform arithmetic operations subtraction. The user enters two numbers, selects the subtraction operation, and clicks the Calculate button. The computed result is then displayed on the screen.

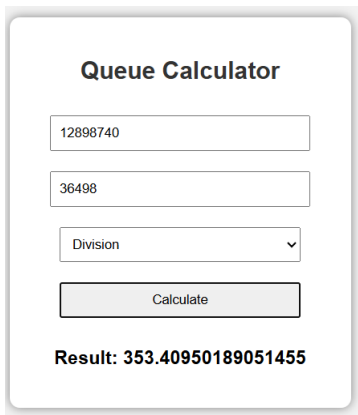
4.1.3 Multiplication operation



The screenshot shows a web-based calculator interface titled "Queue Calculator". It features two input fields: the first contains the number "15" and the second contains "5". Below these is a dropdown menu currently set to "Multiplication". A "Calculate" button is positioned below the dropdown. At the bottom of the interface, the text "Result: 75" is displayed.

Fig 4.1.3: This figure shows a calculator application designed to perform arithmetic operations multiplication. The user enters two numbers, selects the multiplication operation, and clicks the Calculate button. The computed result is then displayed on the screen.

4.1.4 Division operation



The screenshot shows the same "Queue Calculator" interface. The first input field now contains "12898740" and the second contains "36498". The dropdown menu is now set to "Division". The "Calculate" button remains. The result displayed at the bottom is "Result: 353.40950189051455".

Fig 4.1.4: This figure shows a calculator application designed to perform arithmetic operation division. The user enters two numbers, selects the division operation, and clicks the Calculate button. The computed result is then displayed on the screen.

4.2 Discussion

The Simple Calculator project successfully demonstrates how basic arithmetic operations can be integrated with data structure concepts such as queues. By using the queue (FIFO) approach, the system processes user inputs in an organized and sequential manner, improving clarity in understanding data flow. The graphical user interface makes the application interactive and easy to use, even for beginners.

During development, it was observed that implementing queue operations like enqueue and dequeue helps in managing input expressions effectively. The project also highlights the importance of error handling, especially in cases like division by zero or invalid inputs. Overall, the calculator not only performs its primary function but also serves as a practical example of applying theoretical data structure concepts in real-world applications, thereby enhancing programming and problem-solving skills.

CHAPTER 5

CONCLUSION

In conclusion, The Queue-Based Calculator was successfully designed and implemented using HTML, CSS, and JavaScript. The application accepts two numerical inputs and performs basic arithmetic operations such as addition, subtraction, multiplication, and division. The concept of the queue data structure was effectively applied, where the input values are stored using the enqueue operation and processed using the dequeue operation following the First In First Out (FIFO) principle. This ensures that the inputs are handled in the correct order. The system provides accurate results and a user-friendly interface for easy interaction. Overall, this project demonstrates the practical implementation of a queue data structure in a real-time application and helps in understanding its working in a simple and effective manner.

REFERENCES

WEBSITES:

- www.geeksforgeeks.org/queue-data-structure
- www.simplilearn.com/tutorials/data-structure-tutorial/queue-in-data-structure
- www.tutorialspoint.com/data_structures_algorithms/ds_queue.htm

REFERENCE BOOKS:

- Data Structures: From Arrays to Priority Queues – Wayne Amsbury
- Queue Structures and Algorithms – Richard Johnson

APPENDIX

```
<!DOCTYPE html>

<html>

<head>

  <title>Queue Calculator</title>

  <style>

    body {

      font-family: Arial;

      text-align: center;

      margin-top: 50px;

      background-color: #f2f2f2;

    }

    .box {

      background: white;

      padding: 20px;

      width: 300px;

      margin: auto;

      border-radius: 10px;

      box-shadow: 0 0 10px gray;

    }

    input, select, button {

      margin: 10px;

      padding: 8px;

      width: 80%;

    }

    h2 {

      color: #333;
```

```
}  
  
</style>  
  
</head>  
  
<body>  
  
<div class="box">  
  <h2>Queue Calculator</h2>  
  <input type="number" id="num1" placeholder="Enter first number">  
  <input type="number" id="num2" placeholder="Enter second number">  
  <select id="operator">  
    <option value="+">Addition</option>  
    <option value="-">Subtraction</option>  
    <option value="*">Multiplication</option>  
    <option value="/">Division</option>  
  </select>  
  <button onclick="calculate()">Calculate</button>  
  <h3 id="result">Result: </h3>  
</div>  
  
<script>  
  // Queue implementation  
  let queue = [];  
  function enqueue(x) {  
    queue.push(x);  
  }  
  function dequeue() {  
    return queue.shift();  
  }  
  function calculate() {  
    queue = []; // reset queue
```

```
let n1 = parseFloat(document.getElementById("num1").value);
let n2 = parseFloat(document.getElementById("num2").value);
let op = document.getElementById("operator").value;
// Store in queue
enqueue(n1);
enqueue(n2);
// Retrieve from queue
let a = dequeue();
let b = dequeue();
let result;
switch(op) {
    case '+': result = a + b; break;
    case '-': result = a - b; break;
    case '*': result = a * b; break;
    case '/': result = b !== 0 ? a / b : "Error"; break;
}
document.getElementById("result").innerHTML = "Result: " + result;
}
</script>
</body>
</html>
```